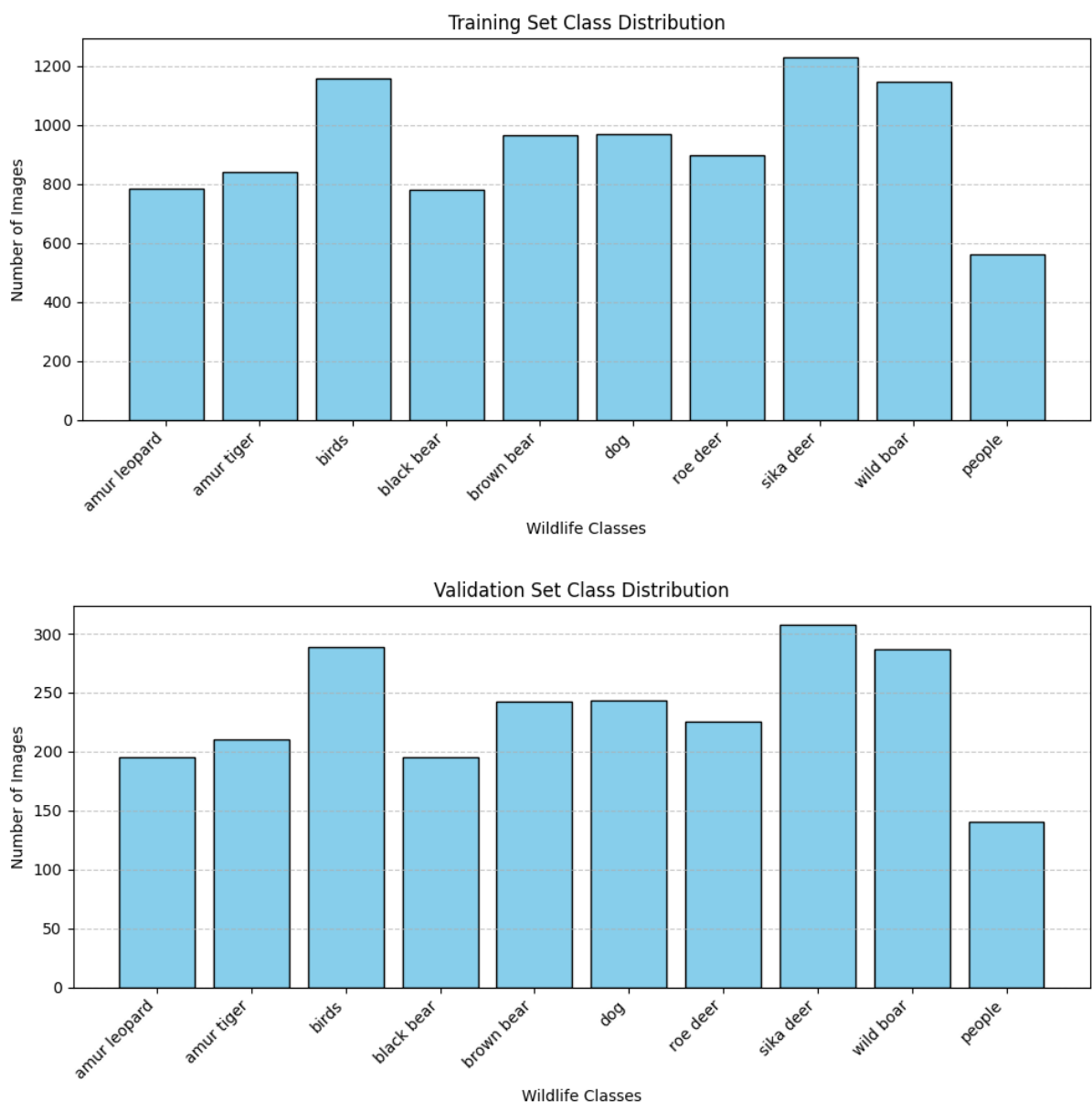


Computer Vision Assignment Report

Name: Saksham Mishra Roll no.: 2023471

1. Image Classification



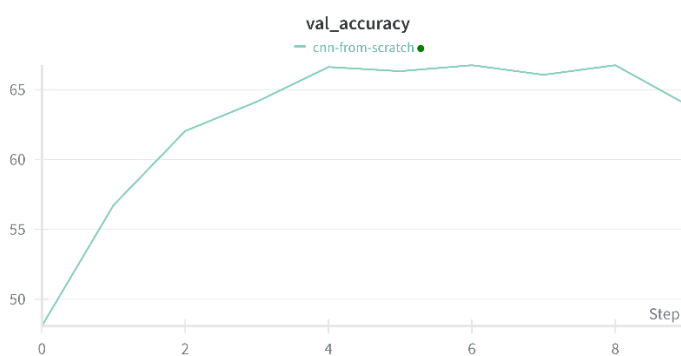
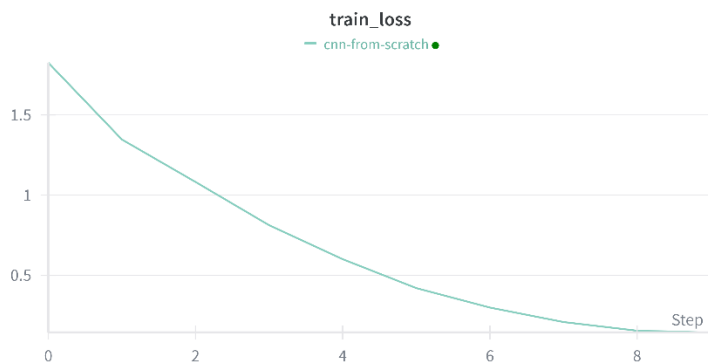
2.2 Training a CNN from scratch

2.2(c) Overfitting Analysis

Conclusion: Yes, the model exhibits significant overfitting to the training dataset.

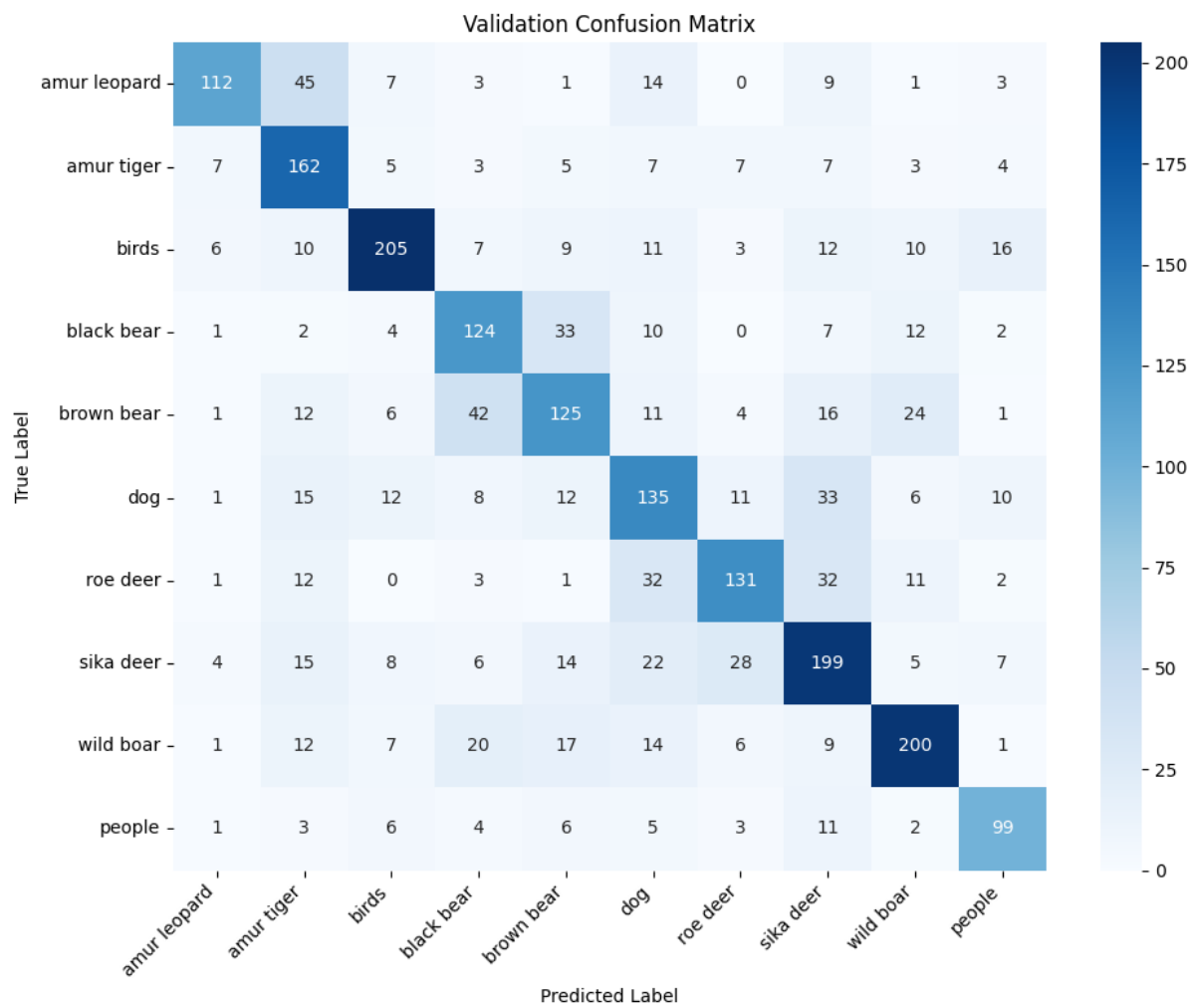
Justification:

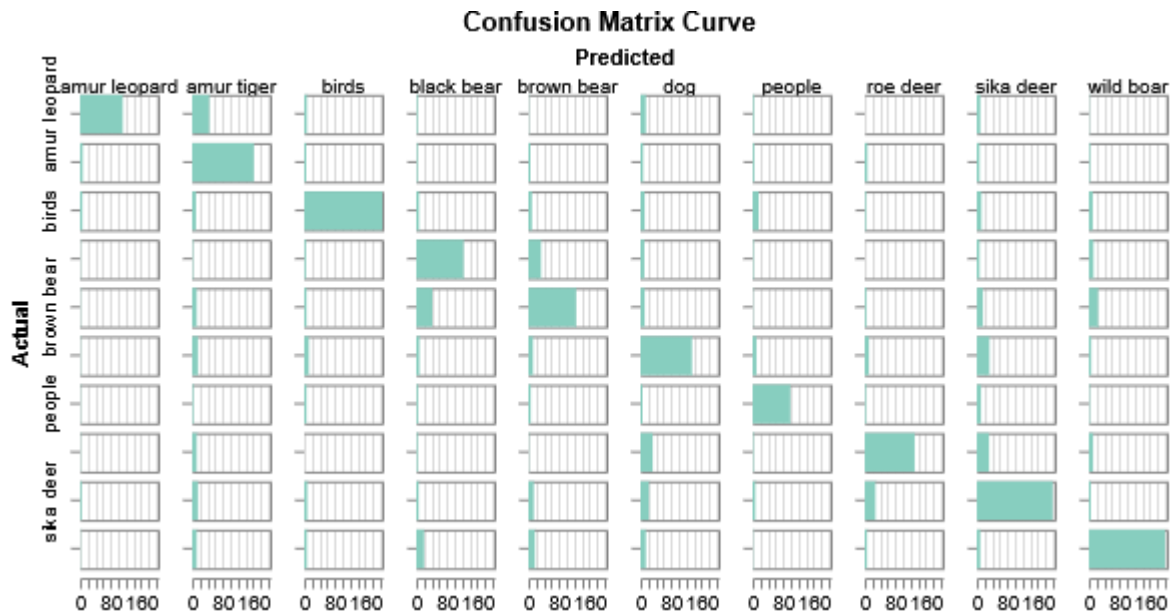
- **Diverging Loss Curves:** During the initial phase (Epochs 1–4), the model demonstrates healthy learning as both training and validation losses decrease. However, a clear "inflection point" occurs at Epoch 4. Beyond this point, the training loss continues its steep descent toward zero (ending at 0.1438), while the validation loss begins a sharp upward trajectory, finishing at 1.8002. This divergence is a classic indicator that the model is no longer generalizing.
- **Accuracy Degradation:** The validation accuracy peaks around Epochs 7–9 at approximately 66.75%. However, in the final epoch, despite the training loss being at its lowest, the validation accuracy drops to 63.92%. This suggests that the model has become so specialized in recognizing the specific noise and backgrounds of the training images that it is now performing worse on unseen data.
- **Generalization Gap:** By the 10th epoch, there is a massive gap between the model's performance on the training set and the validation set. This confirms that the CNN has transitioned from "learning features" to "memorizing samples."



2.2(d) Validation Metrics

- Validation Accuracy: 63.92%
- Validation F1-Score: 0.6406





2.2(e) Analysis of Misclassified Images

- Inter-class Similarity:** The model constantly confuses Black Bears and Brown Bears. Without a sense of scale, their quadrupedal body shapes and dark silhouettes are nearly identical to a shallow CNN. Similarly, the Amur Leopard is heavily misclassified as an Amur Tiger. In infrared/grayscale trail camera shots, the model struggles to differentiate between a leopard's rosettes and a tiger's stripes. Roe Deer and Sika Deer are also repeatedly confused due to identical habitats and body structures.
- Camouflage and Environmental Blending:** People wearing heavy woodland camouflage are predicted as "Dog." Because the camo effectively breaks up the human outline to match the trees, the model likely relies on the background textures and guesses a common woodland animal. A dark wild boar resting in the shadows is reduced to a "dark blob," which the model easily confuses with a brown bear.
- Partial Views and Motion Blur:** In an image where only a blurry tiger tail is visible, the model guessed "Bird" because it saw a fast-moving object suspended in the foliage. For an image showing only the legs of a Sika Deer (predicted as Roe Deer), the model failed because it has not learned the holistic concept of a deer, only patches of pixels.
- Workarounds:** 1. *Grayscale Augmentation:* Train with aggressive grayscale augmentation so the model stops relying strictly on color.

2. *Object Detection:* Shift to an Object Detection model (like YOLO) to force localization of the animal first, isolating it from chaotic backgrounds.

3. *Transfer Learning:* Use a deeper, pre-trained network to learn richer textural differences.

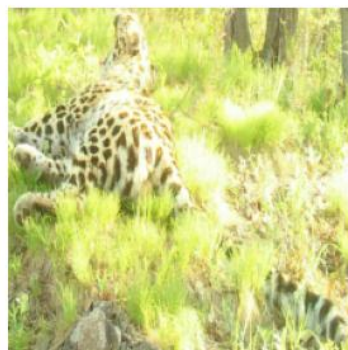
Predicted: amur tiger



True Class: AMUR LEOPARD
Predicted: wild boar



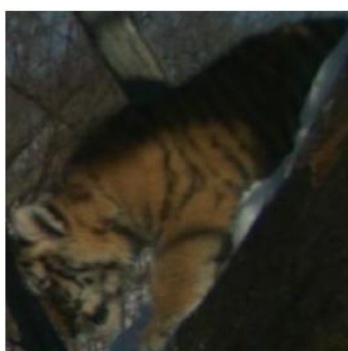
Predicted: birds



Predicted: amur leopard



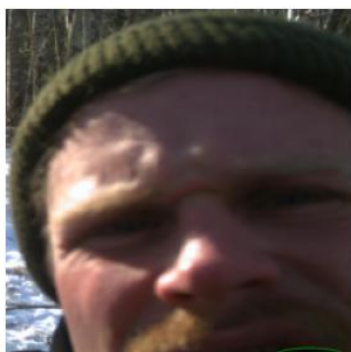
True Class: AMUR TIGER
Predicted: roe deer



Predicted: amur leopard



Predicted: sika deer



True Class: PEOPLE
Predicted: roe deer



Predicted: sika deer



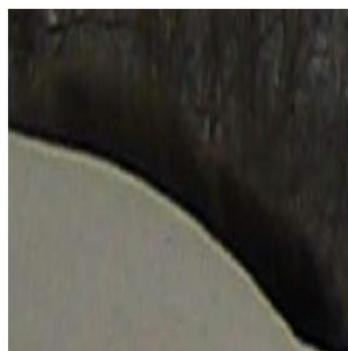
Predicted: black bear



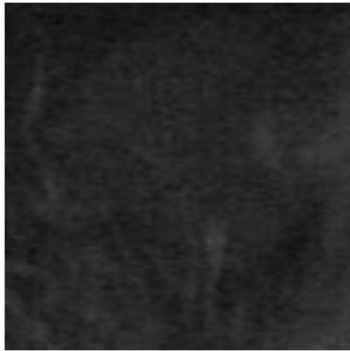
True Class: WILD BOAR
Predicted: sika deer



Predicted: birds



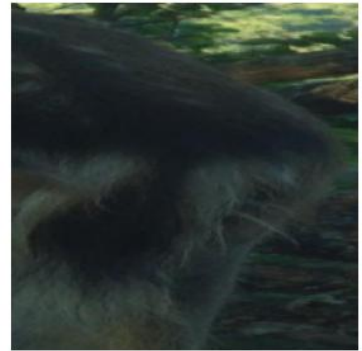
Predicted: amur tiger



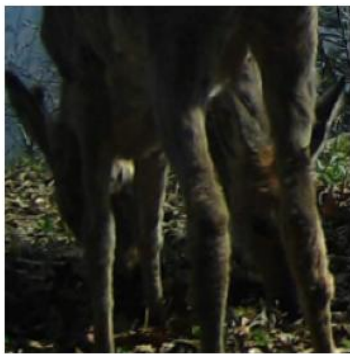
True Class: SIKA DEER
Predicted: amur tiger



Predicted: brown bear



Predicted: sika deer



True Class: ROE DEER
Predicted: sika deer



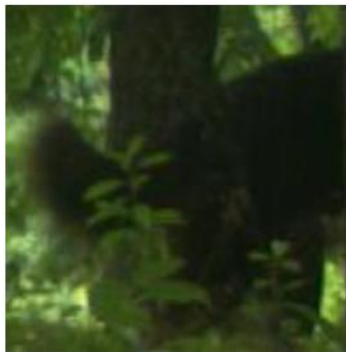
Predicted: sika deer



Predicted: black bear



True Class: DOG
Predicted: black bear



Predicted: roe deer



Predicted: amur tiger

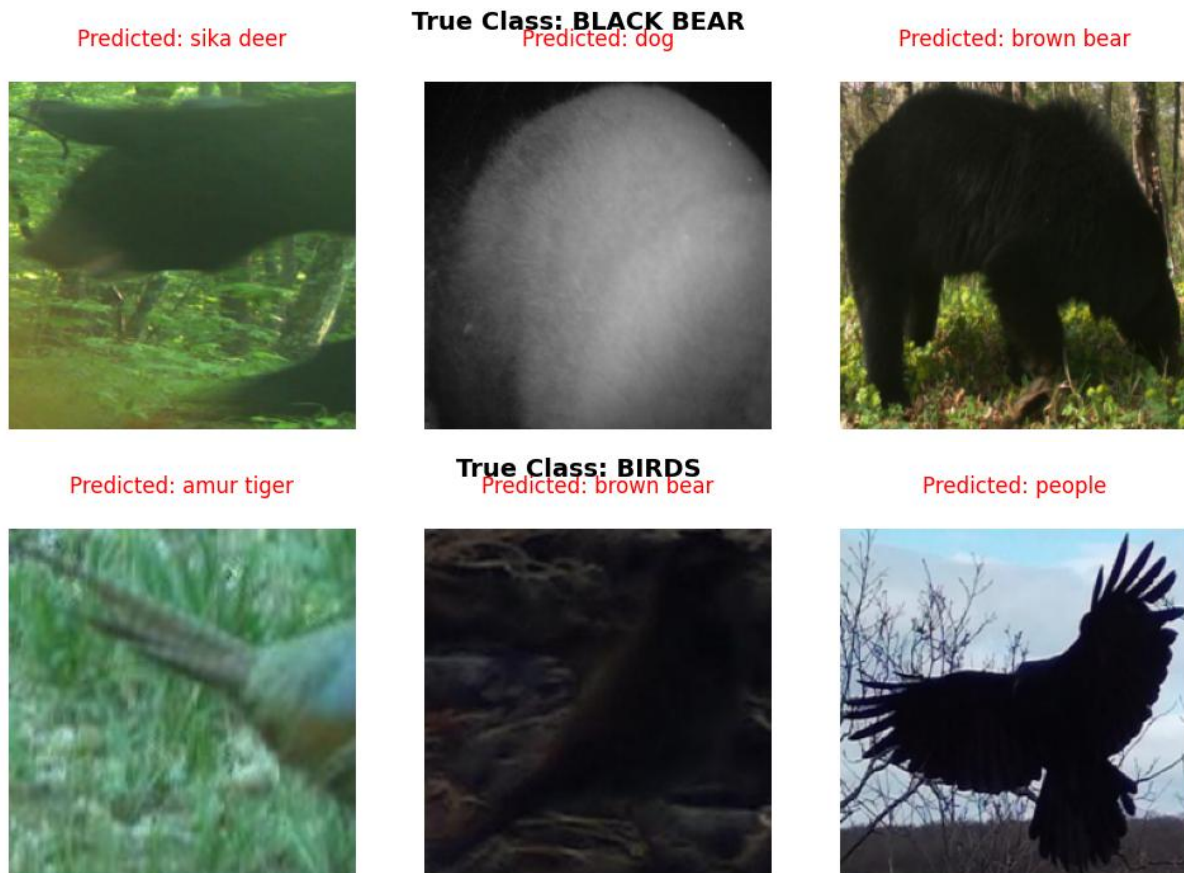


True Class: BROWN BEAR
Predicted: black bear



Predicted: black bear





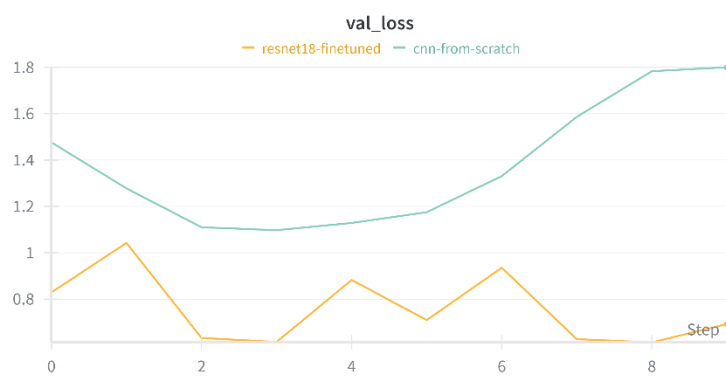
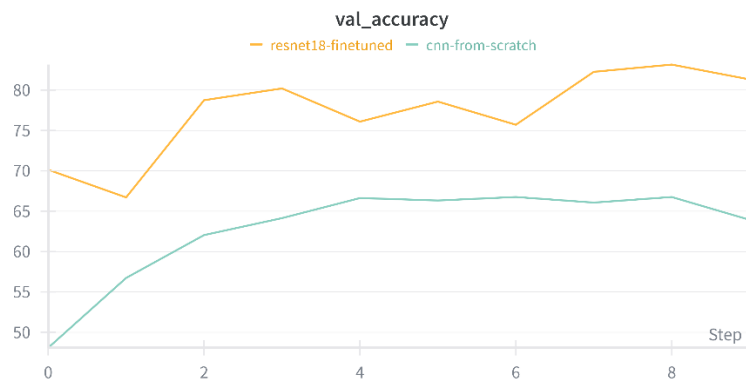
2.3 Fine-tuning a pretrained model

2.3(b) ResNet-18 Overfitting Analysis

Conclusion: The model shows signs of mild overfitting, but demonstrates significantly better generalization and robustness than the Custom CNN.

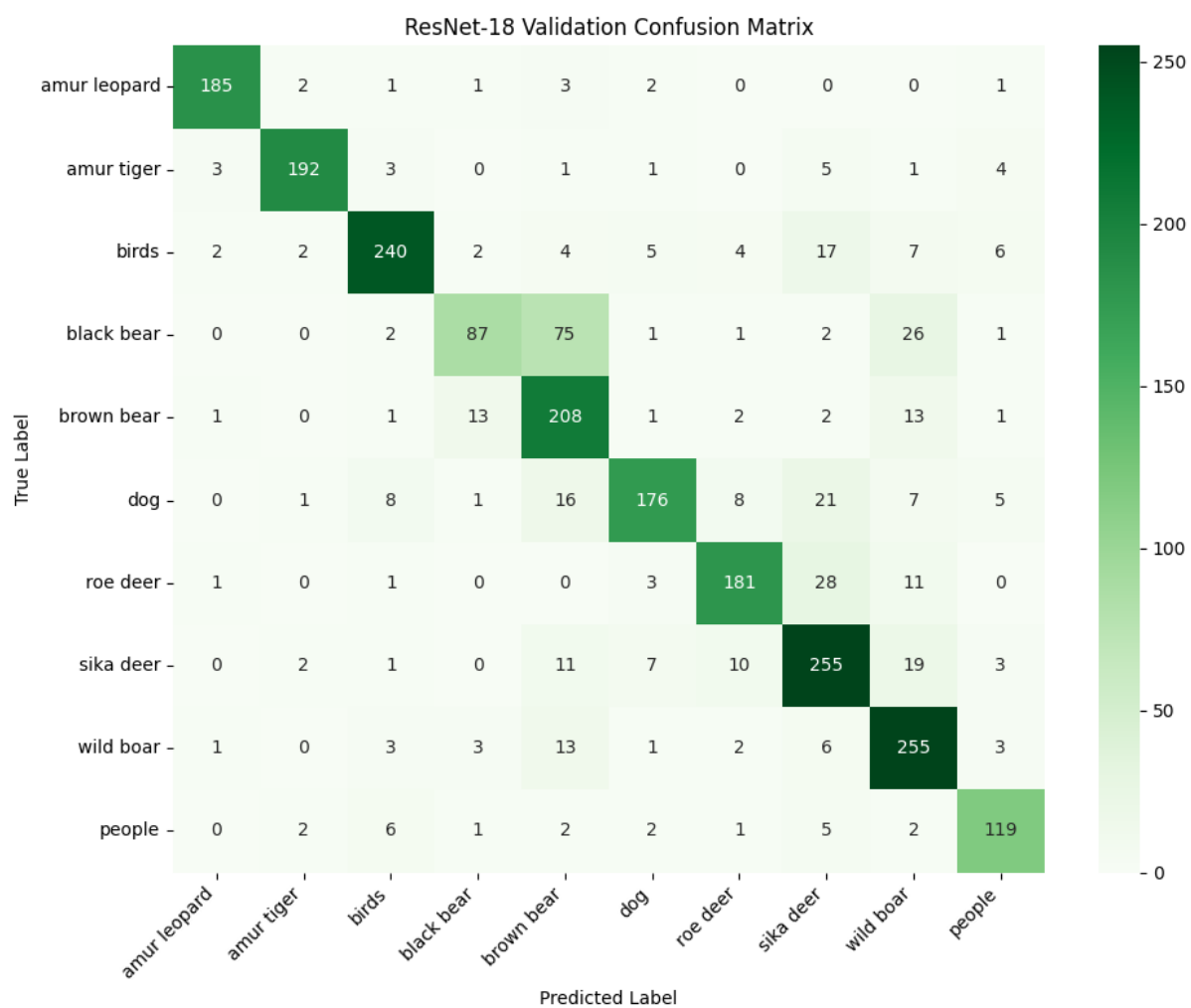
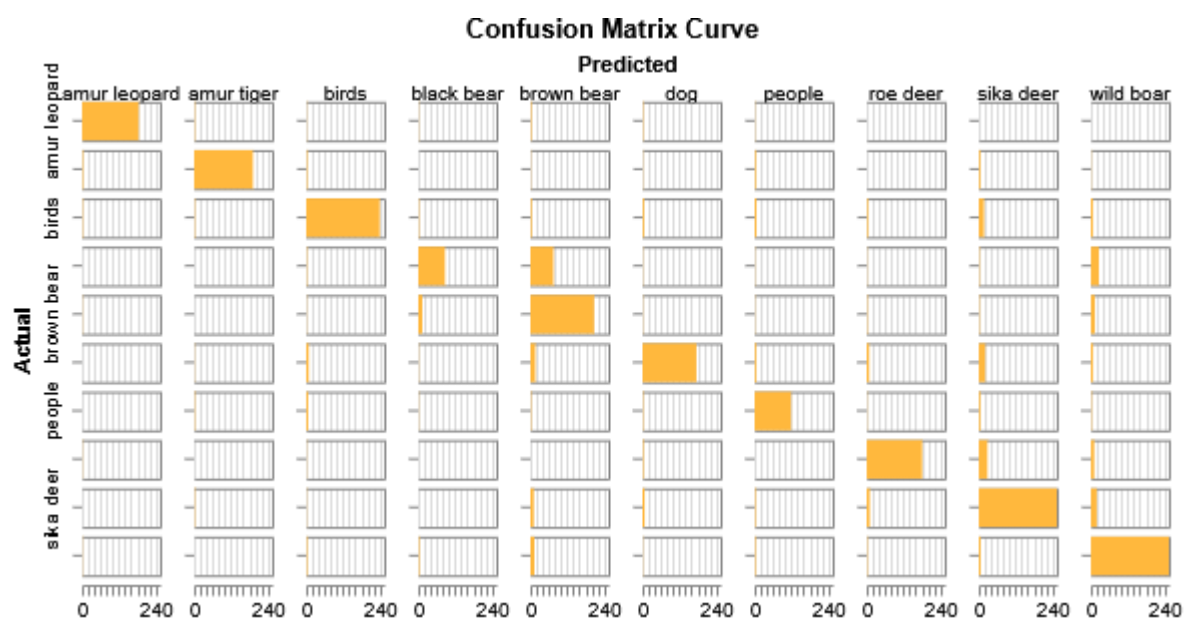
Justification:

- **Loss Curve Behaviour:** The validation loss reaches its global minimum at Epoch 9 (0.6134). While there is a slight increase in the very last epoch (0.6927), the overall trend remains much more stable than the scratch model. The training loss consistently decreases to 0.1337.
- **Accuracy Stability:** Unlike the scratch model, the ResNet-18 continues to improve, peaking at 83.16% in Epoch 9. The final accuracy of 81.32% shows that even when the loss fluctuates slightly, classification capability remains high.
- **Generalization Comparison:** The difference between training and validation performance is much smaller here. The model leverages its pre-trained ImageNet features to understand wildlife, confirmed by a strong F1-Score.

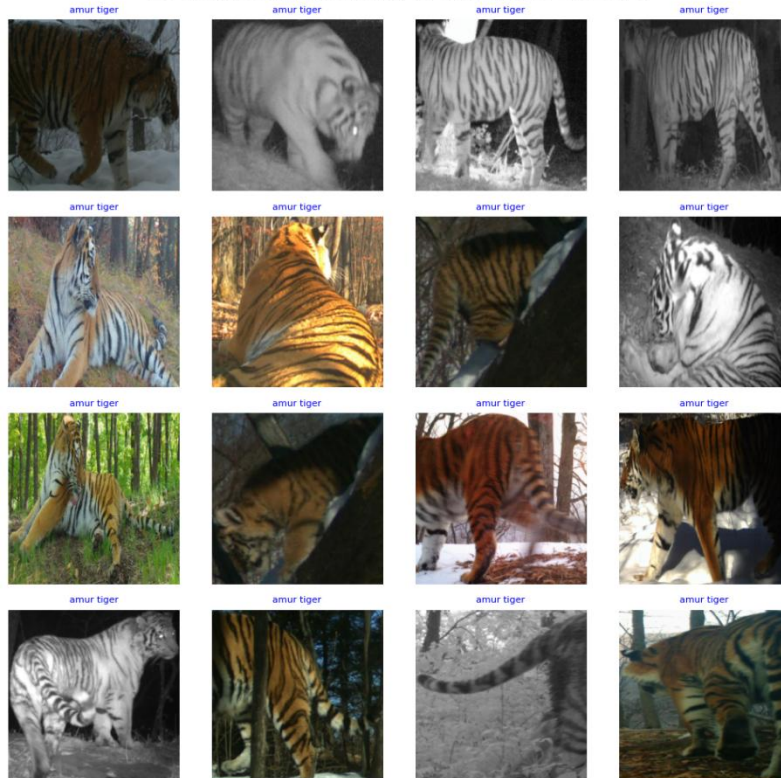


2.3(c) Validation Metrics

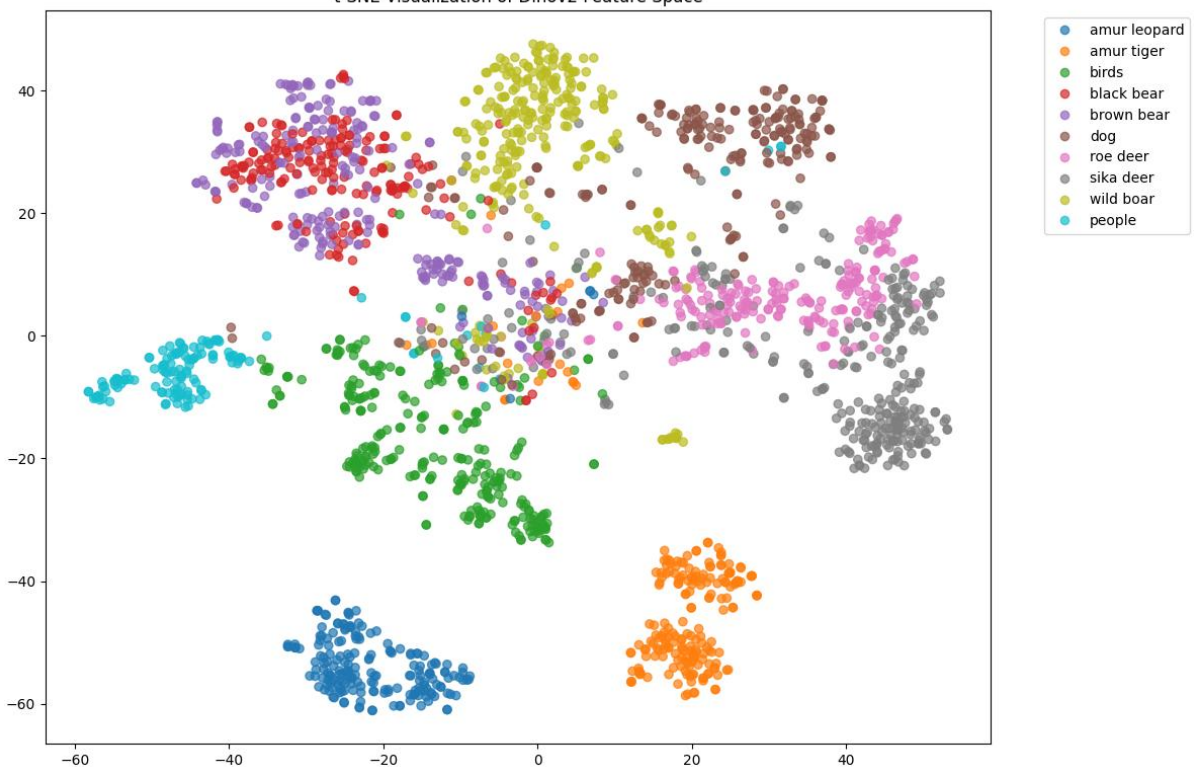
- **Validation Accuracy: 81.32%**
- **Validation F1-Score: 0.8111**



16 Images Automatically Grouped into Cluster 0



t-SNE Visualization of Dinov2 Feature Space



2.4 Data augmentation techniques

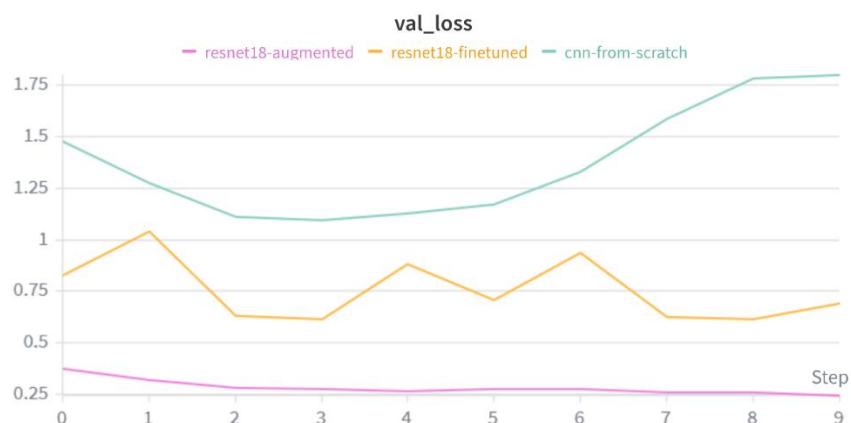


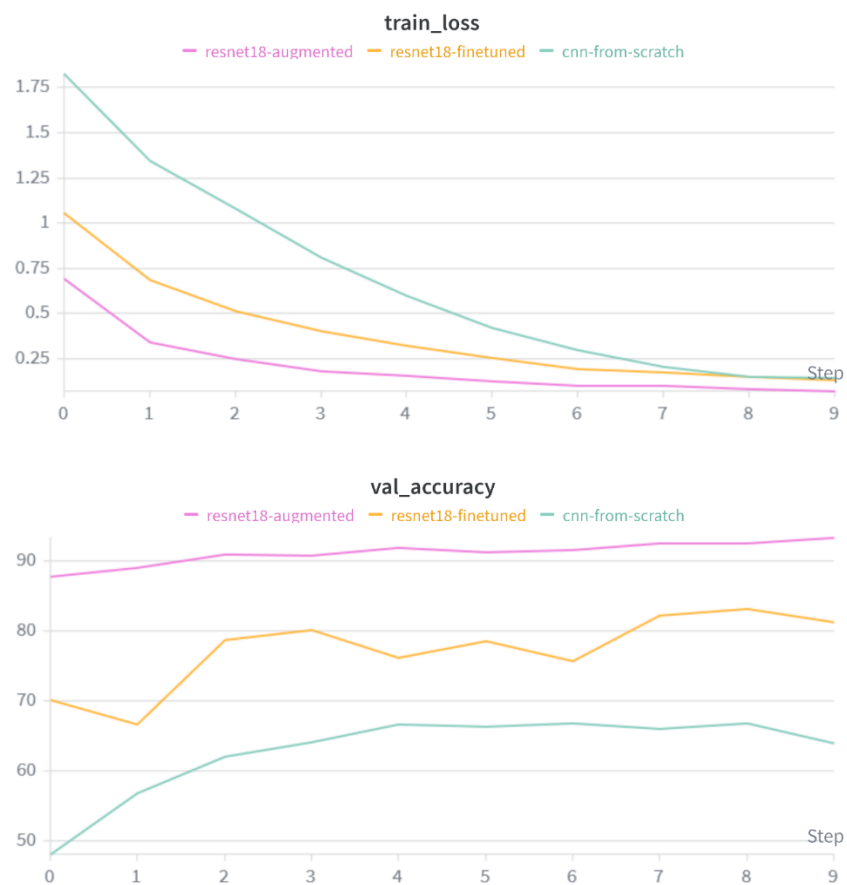
2.4(c) Did Data Augmentation resolve overfitting?

Conclusion: Yes, data augmentation successfully resolved the overfitting problem and drastically improved the model's ability to generalize.

Justification:

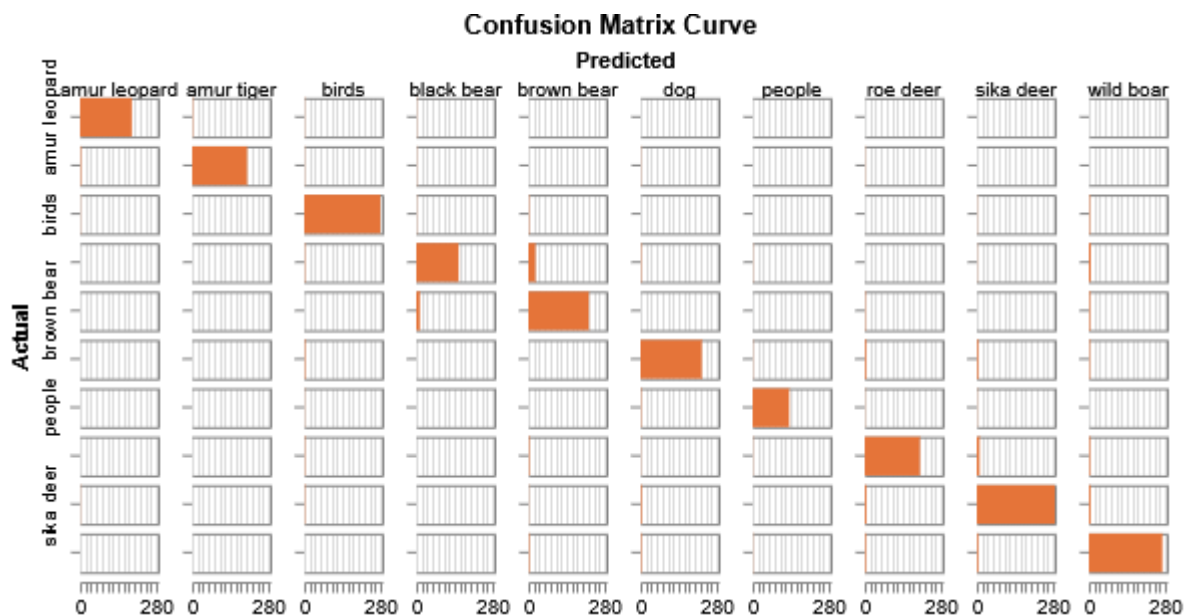
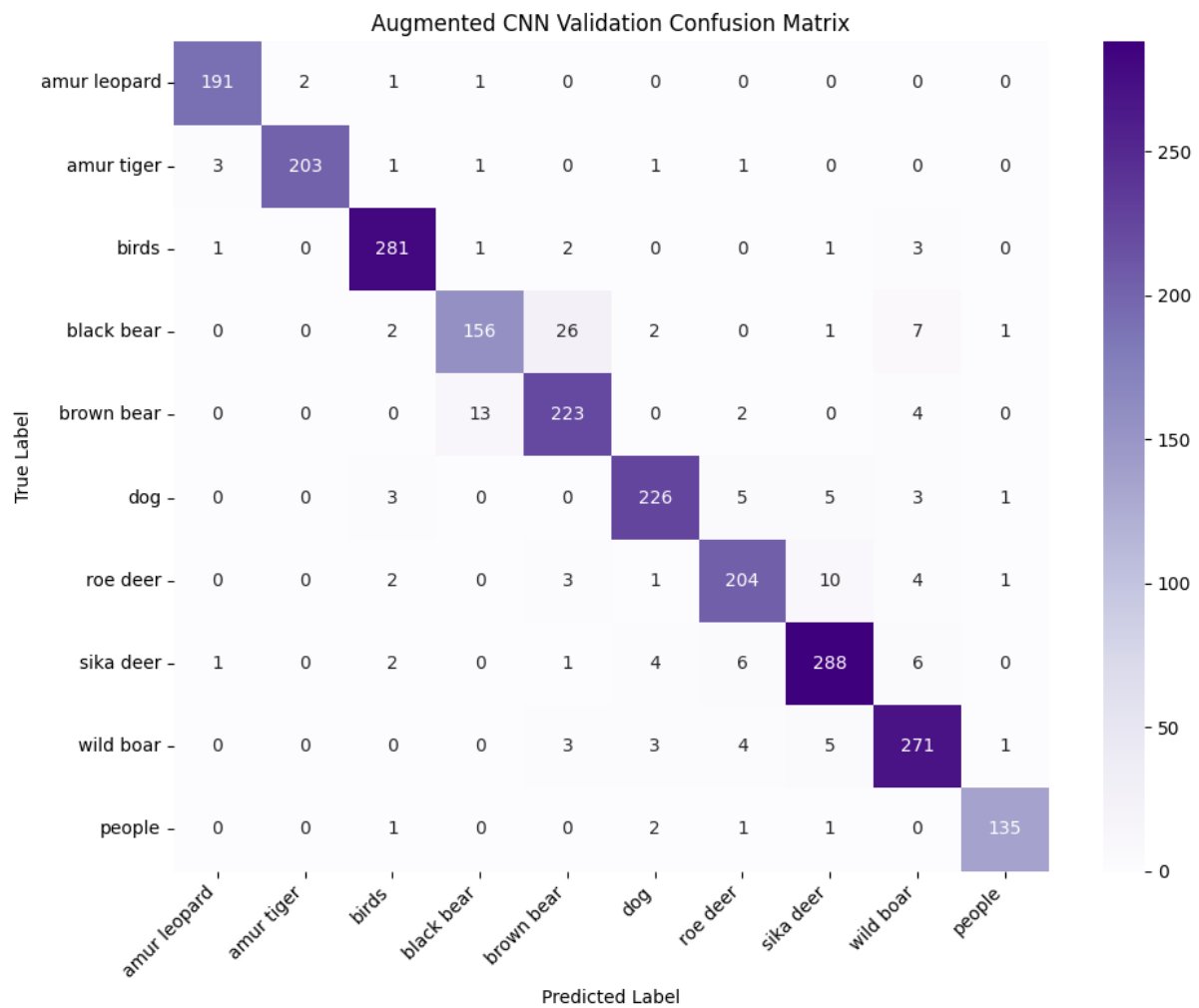
- **The Augmented Improvement:** By applying data augmentation to the ResNet-18 architecture, the validation loss dropped smoothly and stabilized at a highly impressive 0.2442 by Epoch 10, while the training loss ended at 0.0720.
- **The "Why":** While there is a small, healthy gap between the training and validation loss, the validation loss never spiked or diverged. This proves the model successfully generalized. The random transformations (flips, rotations, color shifts) artificially expanded the dataset, preventing the network from relying on exact pixel locations and forcing it to learn robust anatomical features.





2.4(d) Validation Metrics

- **Validation Accuracy:** 93.32%
- **Validation F1-Score:** 0.9329



2.5 Model Comparison and Evaluation

The Winner: Fine-Tuned ResNet-18 with Data Augmentation

The pre-trained ResNet-18 paired with data augmentation is the undeniable winner. It vastly outperformed the baseline Custom CNN and completely eclipsed the unaugmented ResNet-18, proving that transfer learning combined with heavy spatial regularization is the ultimate pipeline for complex wildlife classification.

(a) Convergence behavior and training stability

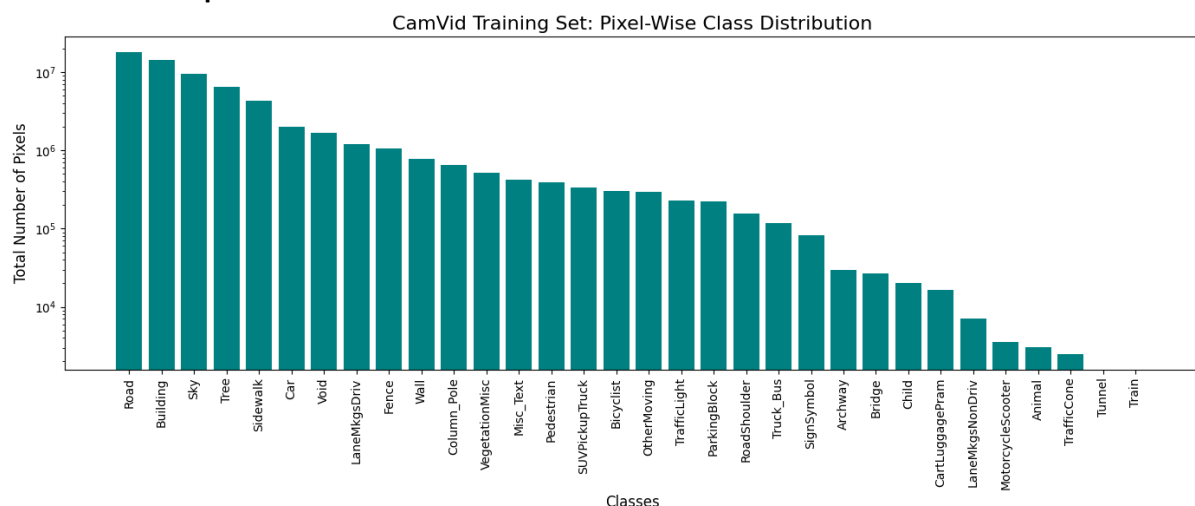
- **CNN from Scratch (No Augmentation):** Exhibited fast convergence on the training data but was highly unstable. By Epoch 5, validation loss skyrocketed while training loss approached zero (severe overfitting).
- **ResNet-18 (No Augmentation):** Converged remarkably fast and remained stable, achieving 81.32% accuracy. However, it still showed a mild divergence between train and validation loss in the final epochs.
- **ResNet-18 with Data Augmentation (Winner):** The augmentations successfully forced the model to learn differently. While the added noise made the training loss drop slightly slower, the validation loss remained incredibly stable, bottoming out at 0.2442. The augmentations acted as the perfect regularizer.

(b) Robustness, variance reduction, and generalization performance

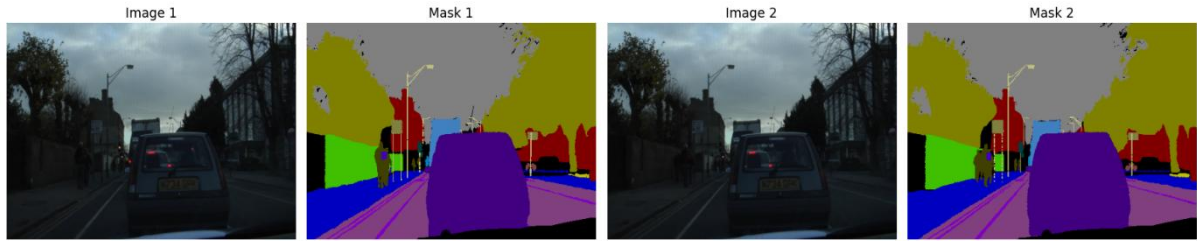
- **Variance Reduction:** The baseline Custom CNN had extremely high variance. Adding Data Augmentation to the ResNet-18 practically eliminated this variance, keeping the validation loss perfectly in check.
- **Generalization Performance:** The Augmented ResNet-18 generalized beautifully, achieving a stellar Validation Accuracy of 93.32% and an F1-Score of 0.9329.
- **Robustness:** The Augmented ResNet-18 demonstrated vastly superior robustness in distinguishing visually similar classes. The deep architecture provided highly tuned edge-and-texture detection, while augmentation forced the model to recognize those textures from any angle, flawlessly separating leopards from tigers and differentiating deer species.

3. Image Segmentation

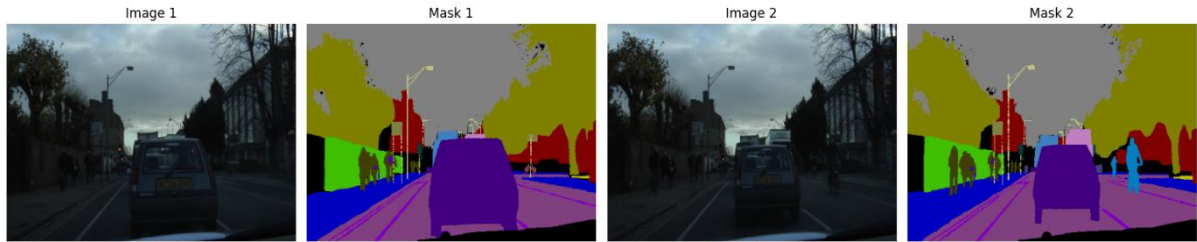
3.1 Dataset Setup



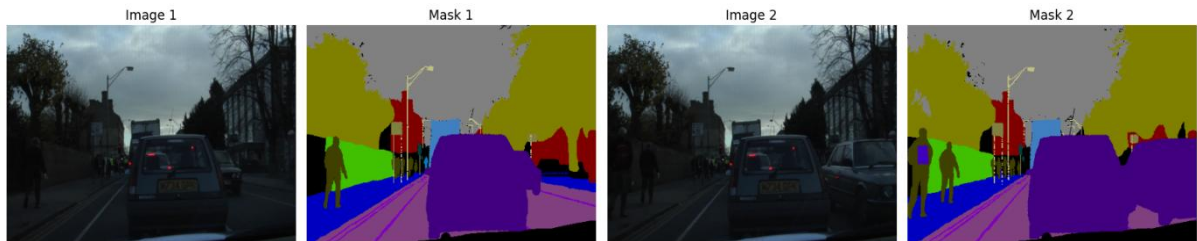
Class: VEGETATIONMISC (Color: [192, 192, 0])



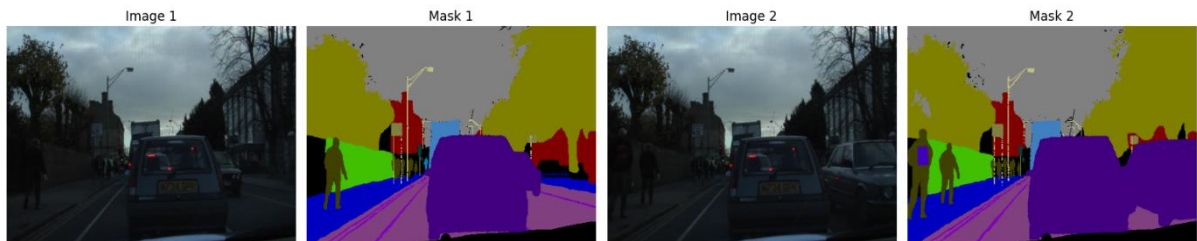
Class: TRUCK_BUS (Color: [192, 128, 192])



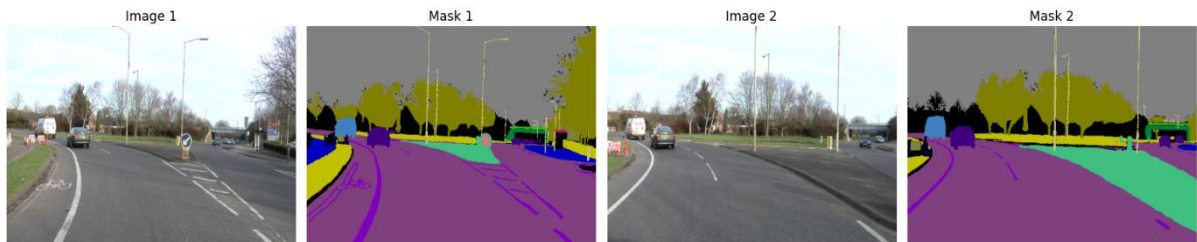
Class: TREE (Color: [128, 128, 0])



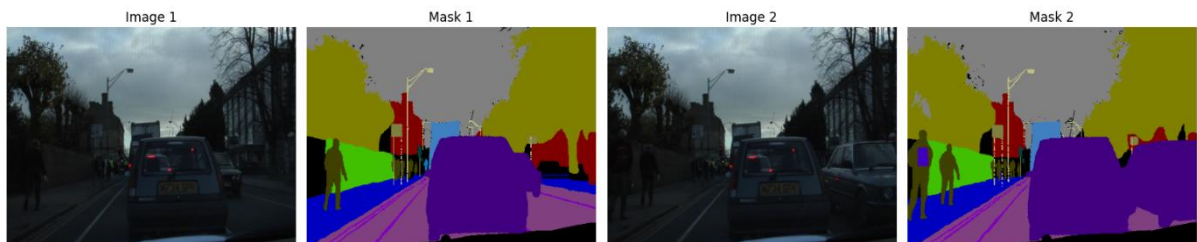
Class: TRAFFICLIGHT (Color: [0, 64, 64])



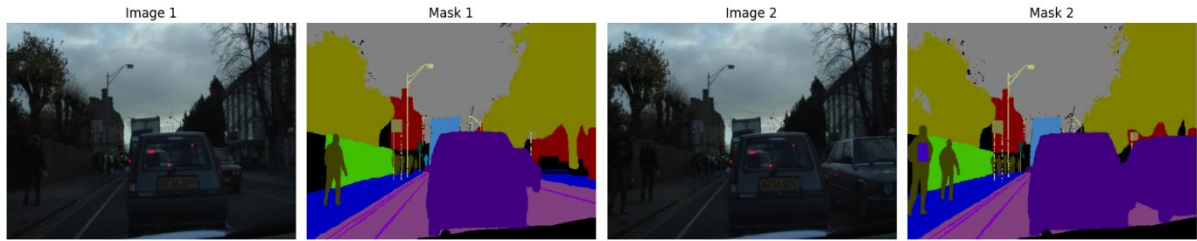
Class: TRAFFICCONE (Color: [0, 0, 64])



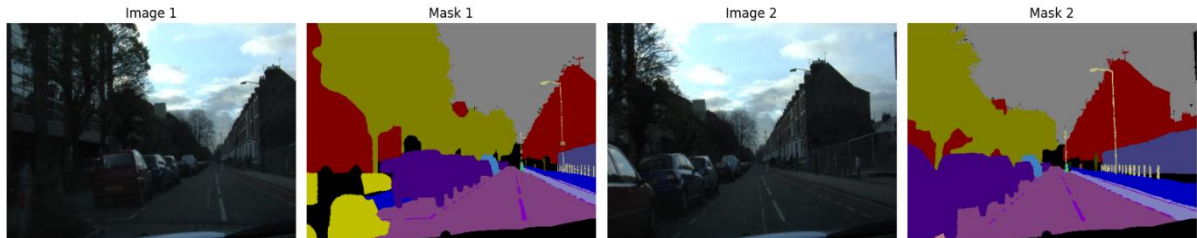
Class: SUV_PICKUPTRUCK (Color: [64, 128, 192])



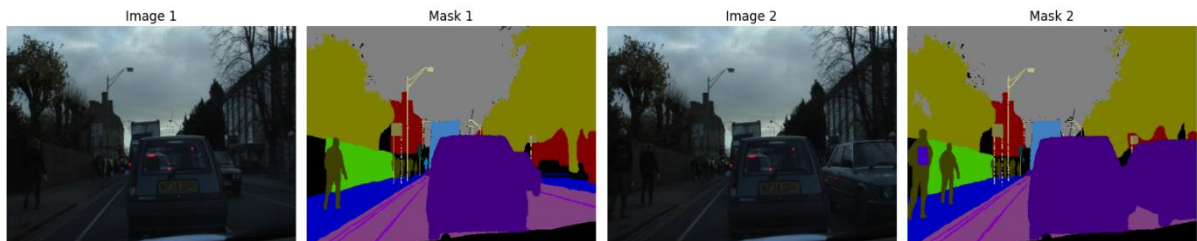
Class: SKY (Color: [128, 128, 128])



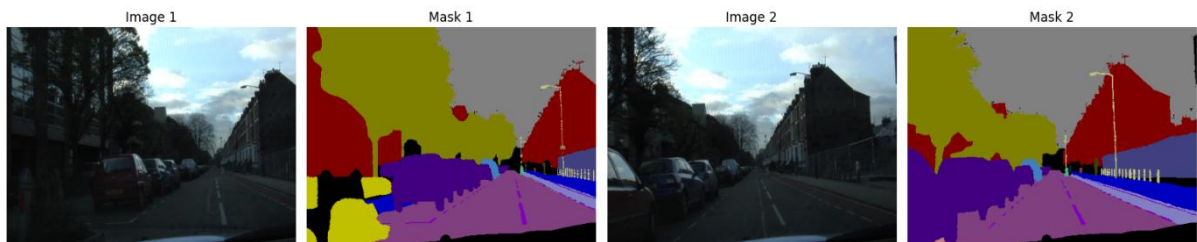
Class: SIGNSYMBOL (Color: [192, 128, 128])



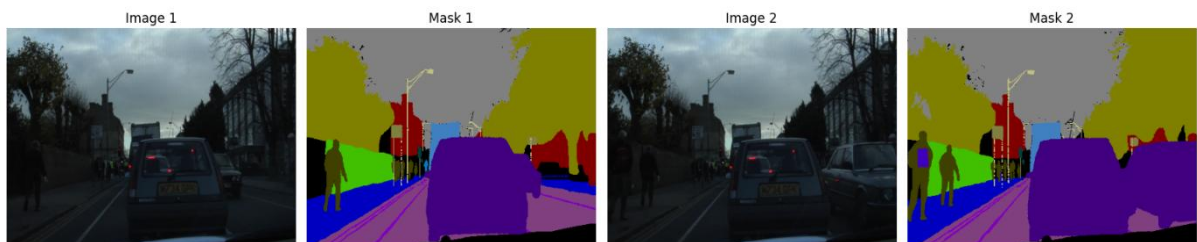
Class: SIDEWALK (Color: [0, 0, 192])



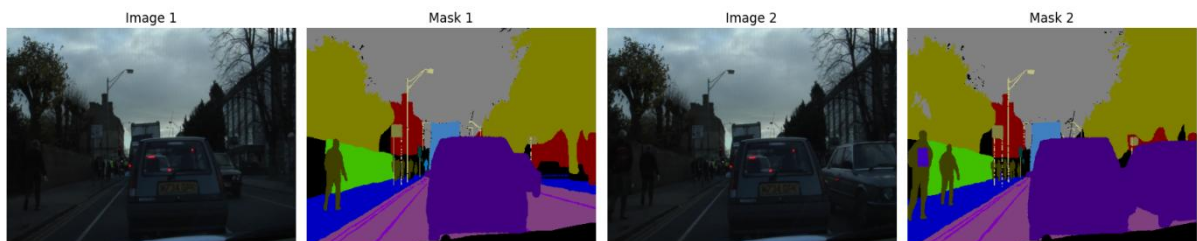
Class: ROADSIDE (Color: [128, 128, 192])



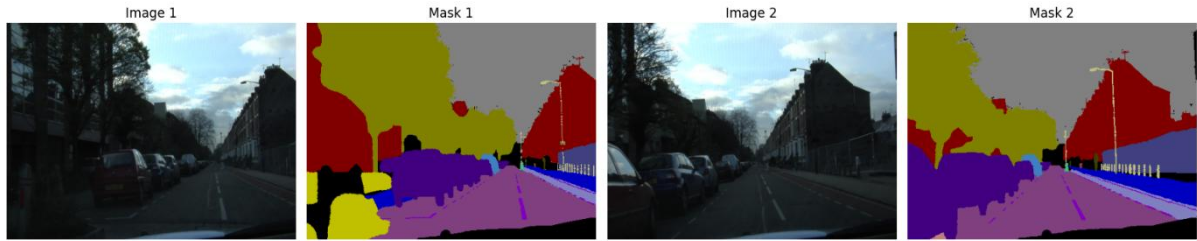
Class: ROAD (Color: [128, 64, 128])



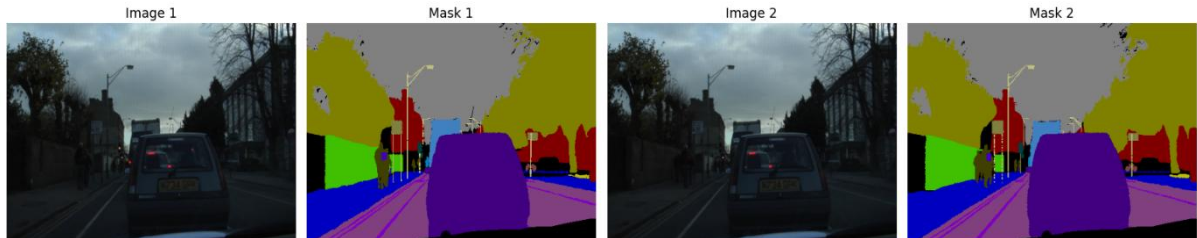
Class: PEDESTRIAN (Color: [64, 64, 0])



Class: PARKINGBLOCK (Color: [64, 192, 128])



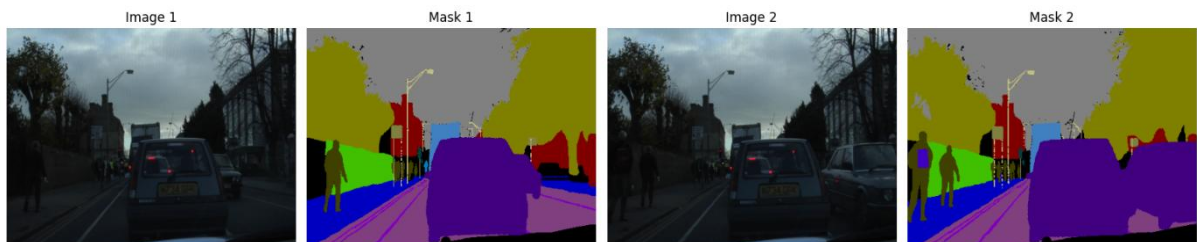
Class: OTHERMOVING (Color: [128, 64, 64])



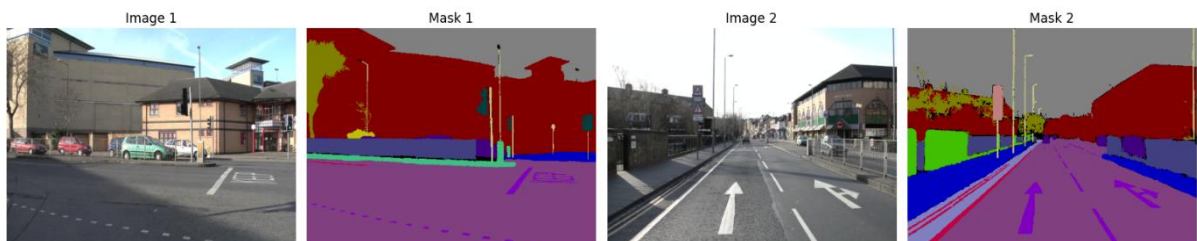
Class: MOTORCYCLESOOTER (Color: [192, 0, 192])



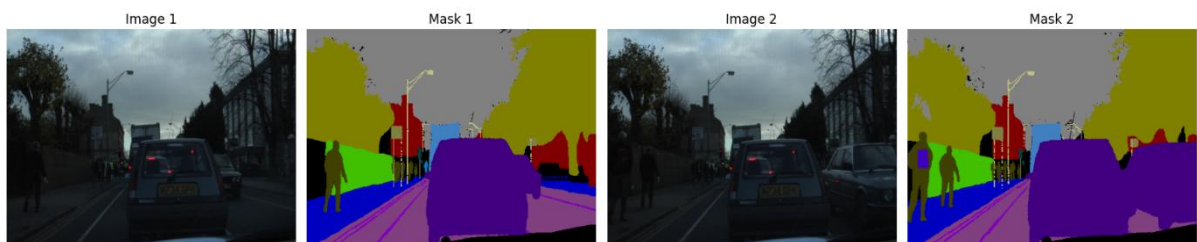
Class: MISC_TEXT (Color: [128, 128, 64])



Class: LANEMKGSNONDRIV (Color: [192, 0, 64])



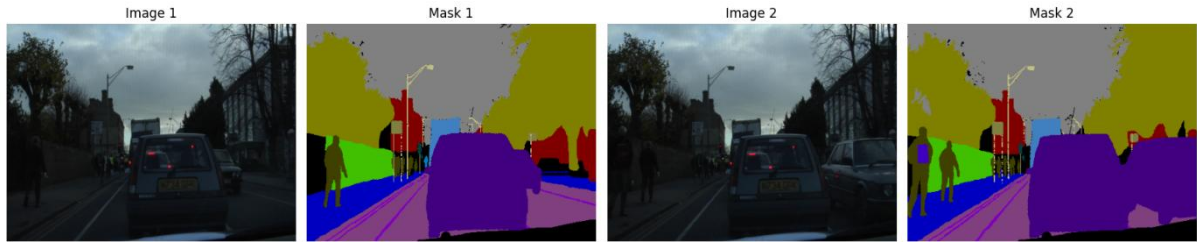
Class: LANEMKGSDRIV (Color: [128, 0, 192])



Class: FENCE (Color: [64, 64, 128])



Class: COLUMN_POLE (Color: [192, 192, 128])



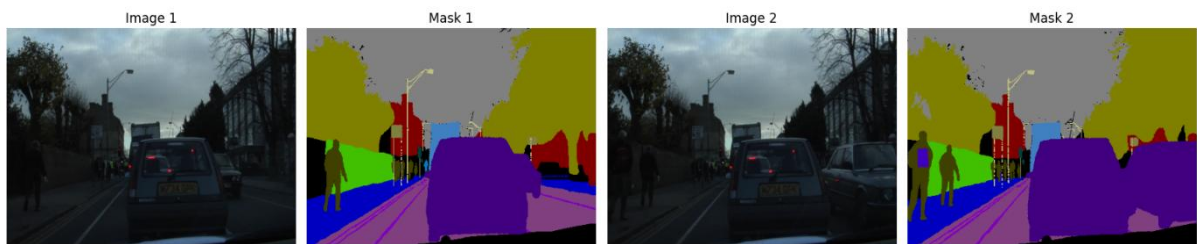
Class: CHILD (Color: [192, 128, 64])



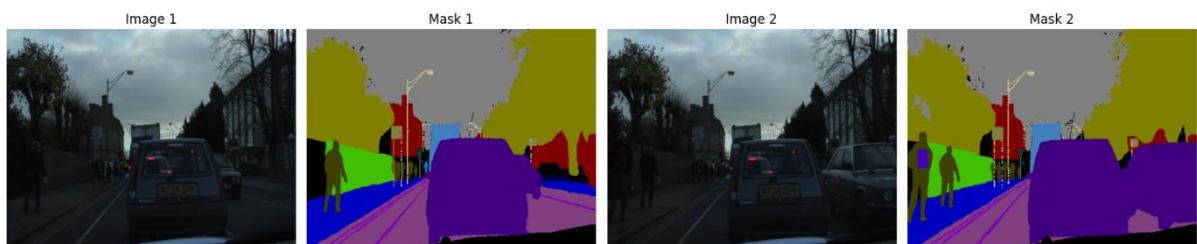
Class: CARTLUGGAGEPRAM (Color: [64, 0, 192])



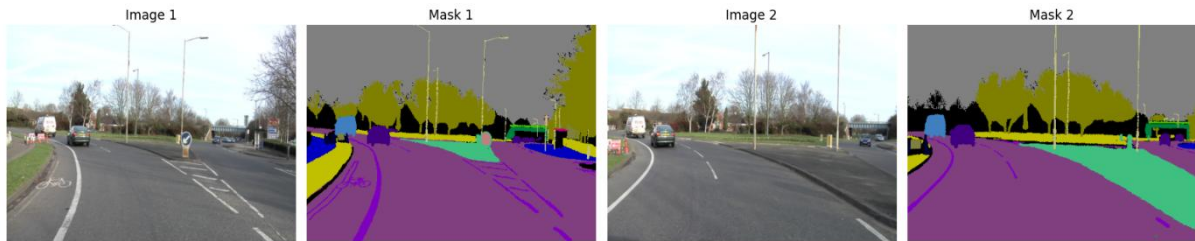
Class: CAR (Color: [64, 0, 128])



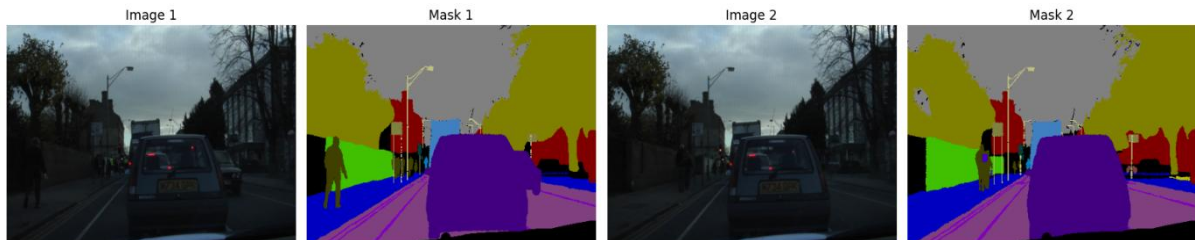
Class: BUILDING (Color: [128, 0, 0])



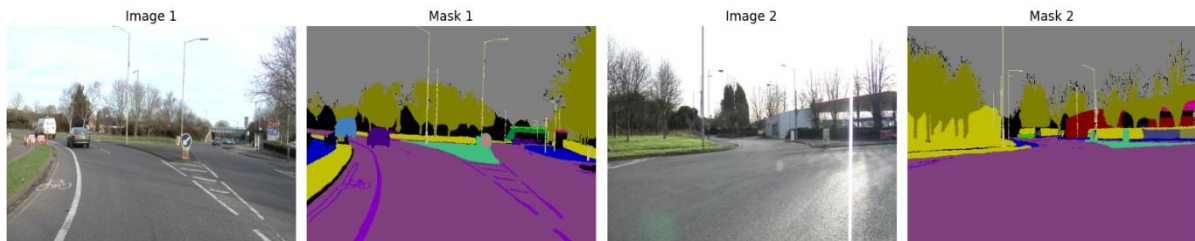
Class: BRIDGE (Color: [0, 128, 64])



Class: BICYCLIST (Color: [0, 128, 192])



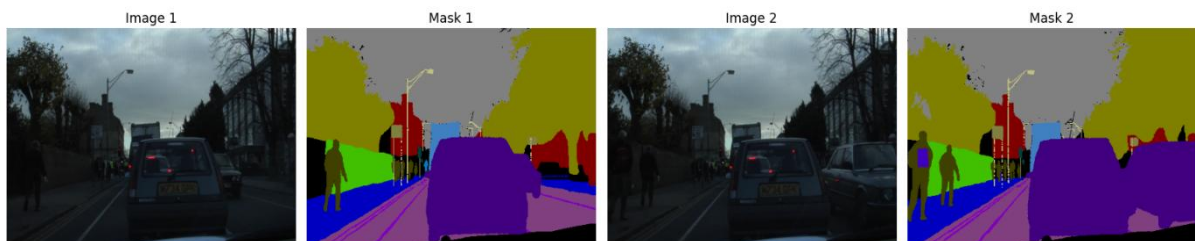
Class: ARCHWAY (Color: [192, 0, 128])



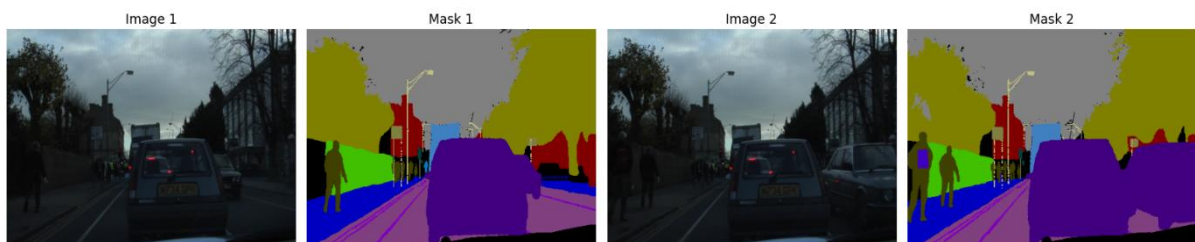
Class: ANIMAL (Color: [64, 128, 64])



Class: WALL (Color: [64, 192, 0])



Class: VOID (Color: [0, 0, 0])

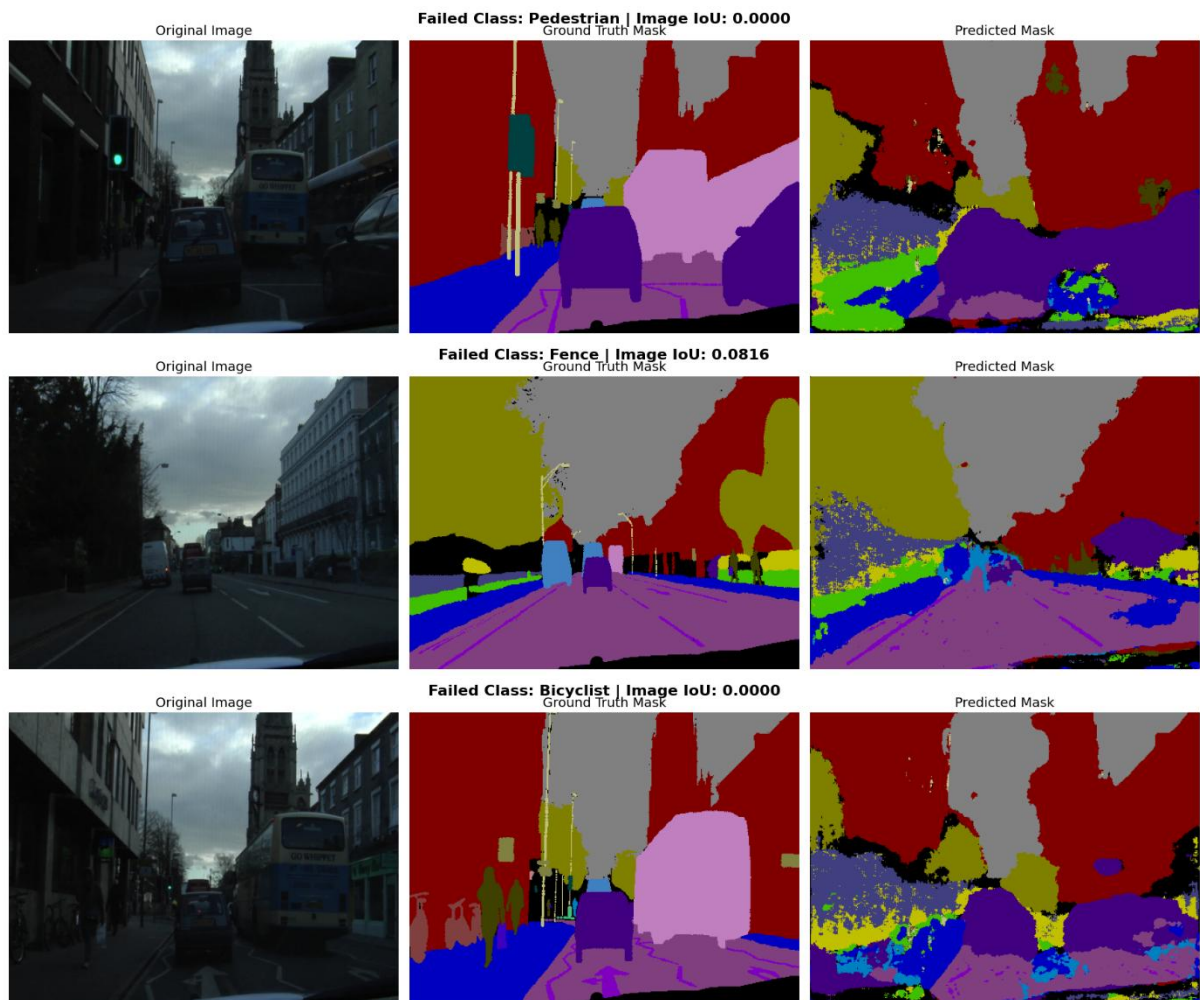


3.2 Train SegNet Decoder from scratch

Final Training Loss: 0.9998 (Epoch 10 Average)

Overall Performance

- **Semantic Pixel Accuracy:** 76.44%
- **Mean Semantic IoU (mIoU):** 0.1827
- **Mean Semantic Dice Coefficient:** 0.2350



3.3 Fine-tuning DeepLabV3

Final Training Loss: 0.3691 (Epoch 10 Average)

Overall Performance

- **Semantic Pixel Accuracy:** 86.58%
- **Mean Semantic IoU (mIoU):** 0.3117
- **Mean Semantic Dice Coefficient:** 0.3946

DeepLabV3 vs. SegNet Comparison

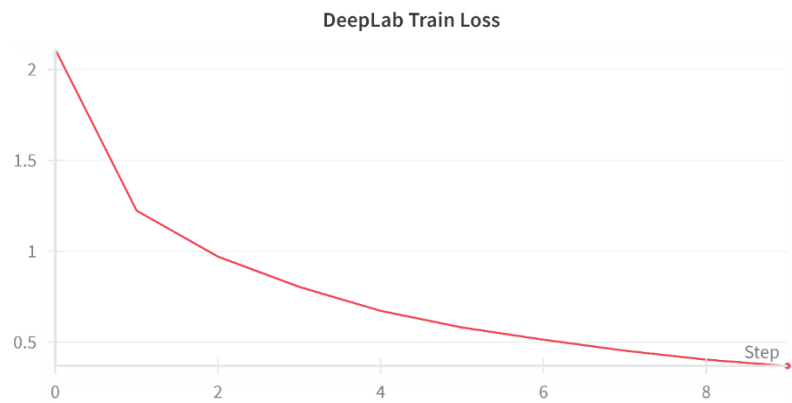
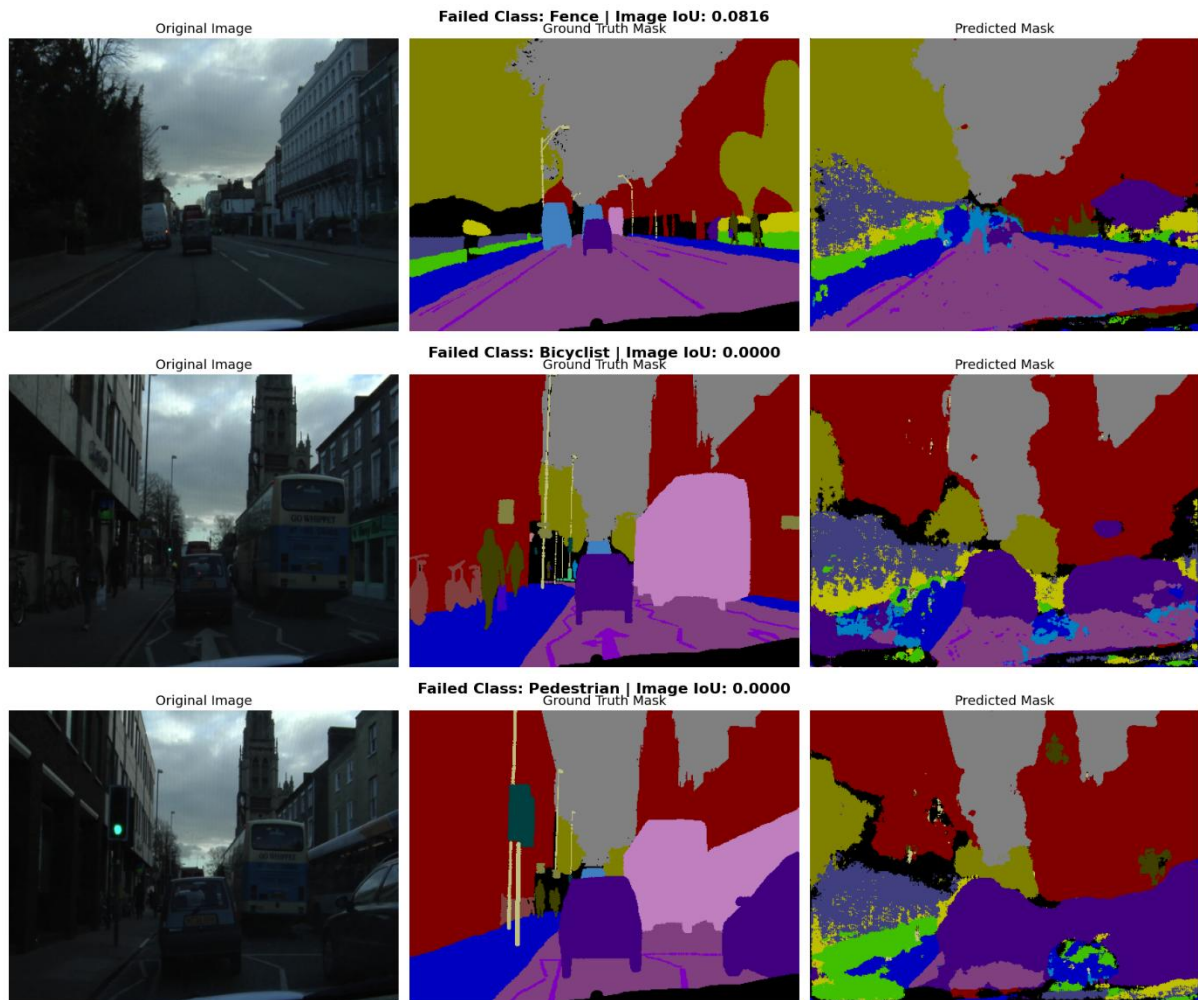
The pre-trained DeepLabV3 vastly outperformed the custom SegNet implementation across all metrics. The SegNet model plateaued with an mIoU of 0.1827, severely struggling with minority classes. In contrast, DeepLabV3 generalized exceptionally well.

- **Architectural Advantage:** SegNet relies on standard convolutions followed by Max-Pooling to compress features, which inherently destroys fine spatial resolution (making it terrible at detecting small objects like poles or pedestrians).
- **DeepLabV3's Secret Weapon:** DeepLabV3 solves this using Atrous (Dilated) Convolutions. This allows the network to have a massive "receptive field" without reducing the actual pixel dimensions of the feature map. Combined with the pre-trained ResNet-50 backbone, DeepLabV3 captures both macro-structures (buildings) and micro-structures (signs) with high precision.

Segmentation Failure Case Analysis (IoU <= 0.5)

Based on the metrics, the models excel at segmenting large contiguous structures (Roads, Skies) but struggle with smaller or highly variable objects.

1. **Pedestrian:** (Environmental Blending & Size). Pedestrians are typically small. The model misses them because dark clothing blends heavily into the surrounding environment (shadows, roads). Repeated downsampling in encoders causes a loss of fine spatial resolution, failing to reconstruct sharp human boundaries.
2. **Bicyclist:** (Misclassification & Geometric Complexity). Bicyclists are a composite of a human and a vehicle. The model confidently segments the human torso as a Pedestrian and the wheels as a MotorcycleScooter, causing the mask to fragment due to heavily overlapping visual features.
3. **Fence:** (Occlusion & Thin Structures). Fences are characterized by complex, thin, vertical lines frequently occluded by vegetation or parked cars. The model fails to predict the continuous structure, predicting broken patches instead, because the "holes" in the fence reveal the background.



4. Object Detection and Multi-Object Tracking

4.1 Ultralytics YOLOv8 on COCO 2017

(b) COCO mAP Performance

- mAP @ 0.50:0.95 (All Areas): 0.313
- mAP @ 0.50 (All Areas): 0.422

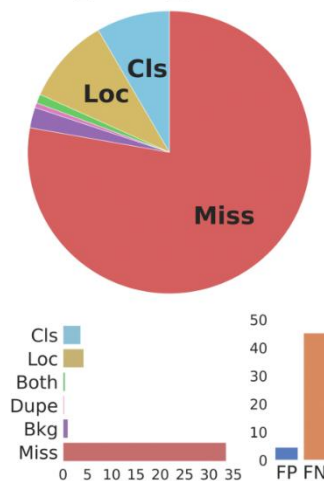
- mAP @ 0.75 (All Areas): 0.345

(c) TIDE Error Analysis

The TIDE analysis reveals that the YOLOv8 model suffers overwhelmingly from False Negatives rather than False Positives. The total False Negative penalty (45.17 dAP) is exactly ten times larger than the False Positive penalty (4.51 dAP).

- **Missed Objects (Miss: 33.56 dAP):** This is by far the most significant bottleneck. The model completely fails to detect a large number of ground truth objects.
- **Localization (4.28 dAP) vs. Classification (3.61 dAP):** When the model detects an object, it struggles slightly more with drawing a perfectly tight bounding box than it does with identifying the class.
- **Background (1.01 dAP) & Duplicates (0.24 dAP):** The model rarely hallucinates objects in empty space, and NMS successfully cleans up duplicate boxes.
- **Conclusion:** The YOLOv8 Nano is a highly precise but low-recall detector. Its primary weakness is a massive blind spot for a large percentage of the dataset.

yolov8_coco_predictions



(d) Expected Calibration Error (ECE)

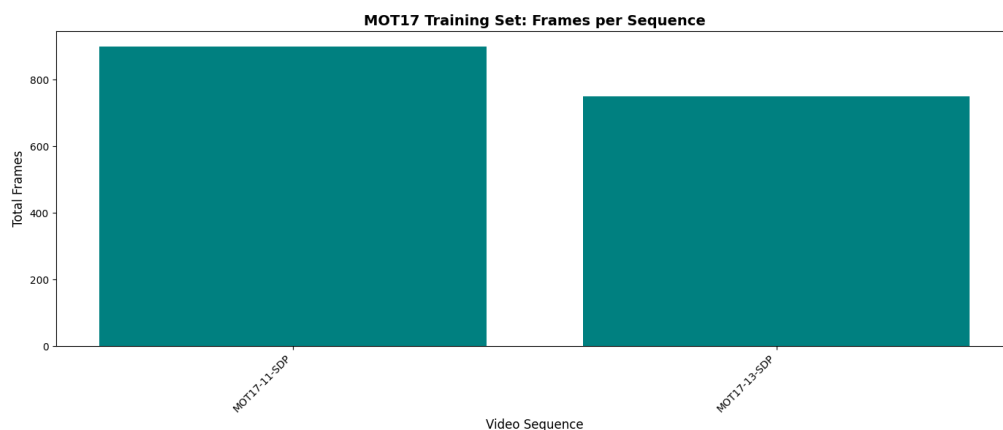
- **Total Predictions Evaluated:** 106,056
- **Expected Calibration Error (ECE):** 0.1479
- **Significance:** Because the ECE is non-zero, it proves the model is overconfident. It frequently assigns high probability scores to bounding boxes that end up being False Positives (or localized poorly). In real-world applications, this miscalibration requires secondary calibration techniques to safely interpret the output probabilities.

(e & f) Scale Analysis

Scale	mAP @ 50	Expected Calibration Error (ECE)	TIDE "Miss" Penalty (dAP)
Small	5.13	0.2486	49.15
Medium	12.92	0.1817	57.85
Large	25.44	0.0939	52.29

- Inferences:** The model's representational capacity struggles linearly with object size. Small objects suffer from severe feature degradation during the pooling layers, leading to high miscalibration and abysmal accuracy. Conversely, large objects preserve rich textural data, allowing the model's confidence to align closely with true accuracy.
- Observation by Scale:**
 - Small Scale:* The model performs worst here (ECE: 0.2486). It is highly overconfident when guessing small objects, and the dominant error is "Miss", meaning it simply cannot see them.
 - Medium Scale:* Performance stabilizes (ECE: 0.1817), but the Miss error actually peaks here at 57.85 dAP, indicating it still leaves a massive amount of ground truth objects undetected.
 - Large Scale:* The model excels here, achieving highly calibrated scores (ECE: 0.0939). Because objects take up substantial pixel space, classification and localization errors become slightly more pronounced relative to smaller scales, as drawing a tight box around massive, irregular objects is difficult.
- Comparison to Global Metrics:** The global metrics (Global ECE: 0.1479) serve as a weighted average that obscures the true scale-dependent weaknesses. Slicing by scale reveals it is dangerously uncalibrated for small objects (0.2486) but highly trustworthy for large objects (0.0939).

Bonus: Multi-Object Tracking (MOT17)



First Frame of MOT17-11-SDP



2(d) Tracker Performance Comparison

When evaluated on the MOT17 training sequence, ByteTrack achieved a Multiple Object Tracking Accuracy (MOTA) of **65.13%**, slightly outperforming the custom-implemented IOU Tracker, which achieved a MOTA of **63.55%**.

- **Limitations of the IOU Tracker:** The custom IOU Tracker relies entirely on spatial overlap (Intersection over Union). While lightweight, it is highly fragile. If an object is temporarily occluded or the detector misses an object for a single frame, the track is lost. Upon reappearance, it assigns a brand-new ID, causing an "Identity Switch" penalty.
 - **The ByteTrack Advantage:** ByteTrack incorporates Kalman Filters to predict the future motion and velocity of objects, allowing it to estimate where an object should be even if detection temporarily fails. Second, it utilizes a dual-association method, storing low-confidence bounding boxes to match with existing tracklets. This allows ByteTrack to maintain consistent tracking IDs through partial occlusions and motion blur.
 - **Conclusion:** The relatively narrow gap between the two scores (~1.58%) suggests that the baseline detections provided for this specific video sequence were of very high quality. However, ByteTrack's motion modeling still proves superior for edge cases involving occlusion.
-

REFERENCES

1. Image Classification & Feature Extraction

- **ResNet-18 Architecture:** He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770-778). Repository: torchvision.models.resnet18
- **DINOv2 (Feature Extraction):** Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., ... & Bojanowski, P. (2023). DINOv2: Learning Robust Visual Features without Supervision. *arXiv preprint arXiv:2304.07193*.

2. Image Segmentation

- **SegNet Architecture:** Badrinarayanan, V., Kendall, A., & Cipolla, R. (2017). SegNet: A deep convolutional encoder-decoder architecture for image segmentation. *IEEE transactions on pattern analysis and machine intelligence*, 39(12), 2481-2495.
- **DeepLabV3 Architecture (Atrous Convolutions):** Chen, L. C., Papandreou, G., Schroff, F., & Adam, H. (2017). Rethinking atrous convolution for semantic image segmentation. *arXiv preprint arXiv:1706.05587*. Repository: torchvision.models.segmentation.deeplabv3_resnet50
- **CamVid Dataset:** Brostow, G. J., Shotton, J., Fauqueur, J., & Cipolla, R. (2008). Segmentation and recognition using structure from motion point clouds. In *European conference on computer vision* (pp. 44-57). Springer, Berlin, Heidelberg.

3. Object Detection & Error Analysis

- **YOLOv8 Model:** Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8. *GitHub repository*. Retrieved from <https://github.com/ultralytics/ultralytics>
- **COCO Dataset:** Lin, T. Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., ... & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. In *European conference on computer vision* (pp. 740-755). Springer, Cham.
- **TIDE (Toolkit for Identifying Errors):** Bolya, D., Foley, S., Hays, J., & Hoffman, J. (2020). TIDE: A General Toolbox for Identifying Object Detection Errors. In *European conference on computer vision* (pp. 558-573). Springer, Cham. Repository: <https://github.com/dbolya/tide>
- **Expected Calibration Error (ECE):** Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In *International conference on machine learning* (pp. 1321-1330). PMLR. (Note: This is the specific paper containing Eqn. 3 referenced in your assignment prompt).

4. Multi-Object Tracking (MOT)

- **ByteTrack:** Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Zeh, Z., ... & Luo, P. (2022). ByteTrack: Multi-object tracking by associating every detection box. In *European conference on computer vision* (pp. 1-21). Springer, Cham. Repository: <https://github.com/ifzhang/ByteTrack>

- **Simple IOU Tracker:** Bochinski, E., Eiselein, V., & Sikora, T. (2017). High-speed tracking-by-detection without using image information. In *2017 14th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS)* (pp. 1-6). IEEE.
- **MOT17 Dataset & Evaluation Metrics:** Milan, A., Leal-Taixé, L., Reid, I., Roth, S., & Schindler, K. (2016). MOT16: A benchmark for multi-object tracking. *arXiv preprint arXiv:1603.00831*. Repository: py-motmetrics (<https://github.com/cheind/py-motmetrics>)